Reconstrucción de identidad vocal mediante TTS *zero-shot* y *fine-tuning* liviano de XTTS v2

Ericson López

***Resumen*—Se estudia en qué medida la *identidad* y el *contenido* de una voz sobreviven al ciclo *voz* → *texto* → *voz*. A partir de una única grabación de reunión (Zoom, español, ∼58 min útiles del locutor principal) se construye de forma automática un corpus (*audio*, *texto*) mediante detección de actividad de voz, filtrado del locutor principal por *embeddings* y transcripción con Whisper. El texto se re-sintetiza con tres motores: XTTS v2 *zero-shot*, XTTS v2 con *fine-tuning* liviano del módulo GPT sobre 45 min del propio locutor, y Bark como *baseline* que no clona. La reconstrucción se evalúa con cuatro métricas complementarias: similitud de locutor (coseno de *d-vectors*), tasa de error de palabra (WER) *round-trip*, distorsión mel-cepstral (MCD) y naturalidad predicha (UTMOS). Finalmente, se discuten las limitaciones (etiquetas ruidosas, sesgo del WER *round-trip*, un solo locutor) y sus implicaciones éticas.**

***Index Terms*—síntesis de voz, clonación de voz, TTS *zero-shot*, XTTS, modelos de lenguaje de audio, *fine-tuning*.**

I. Introducción

La síntesis de texto a voz (TTS) ha pasado en pocos años de sistemas dependientes del locutor, entrenados con horas de audio de estudio, a modelos *zero-shot* capaces de *clonar* una voz nueva a partir de pocos segundos de referencia [1], [2]. Este salto abre aplicaciones legítimas —accesibilidad, doblaje, asistentes personalizados— pero también habilita la fabricación de audio falso (*deepfakes* de voz) con muy poca señal de entrada, lo que motiva estudiar de forma cuantitativa *cuánta* identidad puede realmente recuperarse y bajo qué condiciones.

Este trabajo aborda una pregunta concreta y medible: dado el audio de una persona, ¿cuánta identidad (timbre) y cuánto contenido (inteligibilidad) sobreviven al viaje *voz* → *texto* → *voz*, y cuánto aporta un *fine-tuning* liviano frente al uso *zero-shot* del modelo? A diferencia de los estudios que emplean corpus limpios de estudio, se parte de una grabación *real* de reunión con ruido, compresión y varios interlocutores, un escenario adverso pero realista.

Las contribuciones son: (i) un *pipeline* reproducible que convierte una grabación cruda en un corpus etiquetado (*audio*, *texto*) sin anotación manual; (ii) una técnica de filtrado de locutor principal de audio usando aprendizaje no supervisado; (iii) una comparación controlada entre *zero-shot*, *fine-tuning* liviano y un *baseline* que no clona, con cuatro métricas objetivas complementarias; y (iv) un análisis crítico del compromiso entre fidelidad de timbre e inteligibilidad al afinar sobre datos obtenidos en condiciones naturales (una videollamada de Zoom).

II. Trabajos relacionados

Los sistemas neuronales clásicos, como Tacotron 2 [3], plantean el TTS como una *regresión* de espectrogramas mel a partir del texto, seguida de un vocóder (WaveNet, HiFi-GAN [4]) que convierte el mel en onda. Estos modelos producen voz de alta calidad pero son, en esencia, monolocutor y requieren reentrenar o adaptar con datos abundantes. Una segunda familia, *end-to-end* de una sola etapa, sustituye ese encadenamiento por un autocodificador variacional con flujos normalizadores, alineamiento monótono y entrenamiento adversario: VITS [16] y su extensión multilingüe *zero-shot* YourTTS [17], predecesor directo de XTTS dentro del mismo *toolkit* (Sección III-E) aunque de arquitectura radicalmente distinta.

Un cambio de paradigma reciente trata la síntesis como *modelado de lenguaje* sobre *tokens discretos* de audio. VALL-E [2] demostró que un modelo de lenguaje sobre códigos de un códec neuronal [8] puede clonar un locutor a partir de un *prompt* de 3 segundos. XTTS v2 [1] —el modelo central de este trabajo— sigue esta línea: es un TTS masivamente multilingüe (16 idiomas en el artículo; el *checkpoint* público v2 añade hindi) y *zero-shot* que combina un tokenizador de audio VQ-VAE [5], un *Transformer* autorregresivo tipo GPT-2 [7] condicionado por la voz de referencia mediante un *Perceiver Resampler* [6], y un decodificador HiFi-GAN [4]. Tortoise [9] comparte esta filosofía basada en tokens pero con muestreo más costoso. El *baseline* elegido, Bark [10], genera audio a partir de tokens semánticos y acústicos (EnCodec [8]) pero, en su modo por *presets*, *no* clona la referencia; se emplea precisamente como control de *no-clonación*.

Para el resto del *pipeline* se emplean herramientas preentrenadas de referencia: Whisper [11] para la transcripción robusta, un detector de actividad de voz (Silero VAD [12]) para segmentar, *embeddings* de locutor ECAPA-TDNN [18] para filtrar interlocutores y *d-vectors* entrenados con pérdida GE2E [13] (Resemblyzer) para medir la similitud de voz —dos modelos distintos, de modo que la evaluación no reutiliza el criterio con el que se limpió el corpus. La distorsión mel-cepstral (MCD) [14] es la métrica objetiva clásica de parecido acústico entre dos señales de voz, y UTMOS [19] predice sin referencia la puntuación de naturalidad (MOS) que darían oyentes humanos.

III. Metodología

III-A. Construcción automática del dataset

El preprocesamiento convierte la grabación cruda en cuatro conjuntos con roles distintos (*referencia*, `train`, `val`, `test`)

Tabla I
ESTADÍSTICAS DEL CORPUS CONSTRUIDO (LOCUTOR PRINCIPAL, ESPAÑOL).

Concepto	**Valor**
Fragmentos detectados por VAD	828
Clúster principal (ECAPA)	604
Tras filtro de solape	563
Audio útil del locutor principal	57.8 min
Duración media por fragmento	6.2 s
Referencia de voz (excluida)	4 clips / 38.3 s
`train` / `val` / `test`	442 / 45 / 72
Duración `train` / `val` / `test`	44.6 / 4.6 / 7.9 min

sin anotación manual, en seis etapas:

III-A1. Conversión: La grabación de reunión (`.m4a`) se inspecciona con `ffprobe` y se convierte con `ffmpeg` a WAV mono a 22 050 Hz, la entrada nativa de XTTS.

III-A2. Segmentación: Silero VAD [12] detecta las regiones habladas operando a 16 kHz directamente sobre el audio original (evitando un doble remuestreo) y descarta silencios y pausas. Las regiones más largas que 10 s se dividen en trozos iguales, a cada fragmento se le recortan los silencios de los extremos (`librosa`) y se descartan los menores de 3 s: quedan 828 clips de 3–10 s.

III-A3. Filtrado del locutor principal: La reunión es multi-locutor pero solo interesa el que más habla. Se calcula un *embedding* ECAPA-TDNN [18] por fragmento y se agrupan por *clustering* aglomerativo (distancia coseno, enlace promedio, sin fijar el número de locutores); se utilizó un umbral 0.35 para la tarea no supervisada. El mayor clúster (604 fragmentos) se toma como locutor principal.

III-A4. Filtro de solape: El *clustering* usa un *embedding global* por clip y no detecta mezclas de voces *dentro* de un clip. Por eso se calculan también *embeddings* sobre ventanas deslizantes de 1.2 s (salto 0.75 s): si la similitud mínima entre ventanas cae bajo el umbral 0.06, hay probablemente un cambio de voz dentro del fragmento y se descarta. Al final de este proceso quedan 563 fragmentos (57.8 min) del locutor principal. Se valida el proceso con diagnóstico visual y escucha de muestras, tratando de evitar que fragmentos de múltiples locutores o fragmentos de locutores no principales sean tomados en cuenta como datos para el resto del pipeline.

III-A5. Transcripción: Cada fragmento se transcribe con Whisper `large-v3` [11] (*beam search* de 5), produciendo el par (*audio*, *texto*).

III-A6. Referencia y splits*:* Se construye una *referencia de voz* dedicada concatenando 4 clips largos (4–10 s) de alta similitud al centroide (38.3 s en total). El resto se reparte en `train`/`val`/`test` (80/8/12 %) *por región* de VAD: los trozos de una misma región van al mismo *split*, para que fracciones de una misma frase no crucen de `train` a `test` arrojando métricas sesgadas. La Tabla I resume el corpus.

III-B. El modelo: XTTS v2

XTTS v2 [1] *no* es un TTS clásico (texto → mel → vocóder): es un "modelo de lenguaje de audio" que genera *tokens*

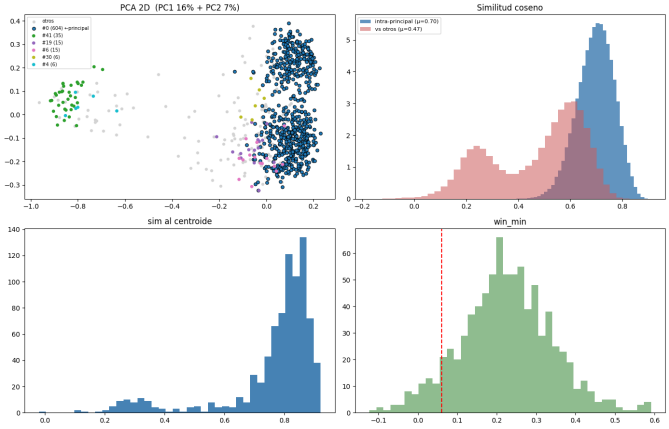


Figura 1. Validación del filtrado del locutor principal. *Arriba-izq.*: PCA 2-D de los *embeddings* ECAPA por *clúster*; el principal tiene borde negro y el resto son los demás interlocutores. *Arriba-der.*: la similitud *intra*-clúster principal se separa de la similitud contra los demás: una sola voz distinta del resto. *Abajo-izq.*: similitud al centroide del locutor. *Abajo-der.*: filtro de solape; bajo el umbral (línea roja) se descarta por posible mezcla de voces.

discretos de sonido de forma autorregresiva. La idea que articula todo el diseño es que, si el audio puede convertirse en una secuencia de símbolos de un vocabulario finito, sintetizar voz se reduce a la misma tarea que resuelve un modelo de lenguaje —predecir el siguiente símbolo— y *clonar* una voz se reduce a condicionar esa predicción con un prefijo extraído de una grabación de referencia, sin tocar un solo peso. De ahí que XTTS necesite módulos auxiliares que traduzcan entre el mundo continuo de la señal y el mundo discreto del transformer: uno que discretiza el audio (DVAE), otros que resumen la voz de referencia y uno que devuelve la señal al dominio de la onda (HiFi-GAN). Consta, pues, de cinco bloques (Fig. 2), de los que solo el GPT se afina en este trabajo:

1. **Tokenizador de texto.** El texto se normaliza por idioma (números, abreviaturas, signos) y se segmenta con BPE a un vocabulario de 6 681 subunidades, anteponiendo un token de idioma que indica al modelo en qué lengua debe pronunciar —el mismo texto genera fonética distinta según ese token. La secuencia se limita a 402 tokens y el GPT la vectoriza con una tabla de *embeddings* y unas posicionales *propias*, separadas de las del audio. El papel de este bloque es, por tanto, poner el texto en el mismo formato simbólico que el audio, de modo que un único transformer *decoder-only* pueda leer ambos flujos en una sola secuencia.

2. **Codificadores de la referencia de voz.** Es el bloque responsable del carácter *zero-shot*: la referencia se codifica *una sola vez* y en *dos* representaciones, porque hay dos decisiones distintas que dependen del locutor. (a) El *latente de condicionamiento*: sobre el mel de 80 bandas de la referencia, un *Conditioning Encoder* de seis capas de atención y un *Perceiver Resampler* [6] —un transformer que cruza un conjunto *fijo* de consultas aprendidas contra la referencia mediante *cross-attention*— producen **siempre** 32 vectores de 1 024-d, con

independencia de la longitud del audio de referencia. Esta secuencia entra sin codificación posicional (no existe orden temporal) como prefijo al GPT. (b) El *embedding de locutor* de 512-d, extraído por un modelo de verificación de locutor (H/ASP) a 16 kHz, que condiciona directamente al vocóder (Hifi-GAN): fija la identidad en el timbre de la forma de onda.

3. **DVAE: el tokenizador de audio.** Un VQ-VAE [5] de 13 M parámetros que cumple con el audio el papel que el BPE cumple con el texto. Recibe el espectrograma mel de 80 bandas calculado sobre el audio a 22.05 kHz con $hop_lenght = 1024$; posteriormente, un codificador convolucional comprime el mel hasta 21.53 Hz, con el fin de que cada paso de tiempo sea un token y se represente en 1 024 muestras del audio original— y cada vector resultante se sustituye por el código más cercano de un *codebook* de 8 192 entradas, filtrado a los 1 024 más frecuentes (+2 símbolos especiales) [1]. La salida es una secuencia de tokens de audio. El bloque DVAE es preentrenado y solamente se utiliza su encoder en fine-tuning/entrenamiento para fabricar las etiquetas (one-hot encoding) con las que el GPT aprende.

4. **GPT-2 autorregresivo.** El núcleo del sistema: un GPT-2 [7] *decoder-only* de 443 M parámetros (30 capas, d_{model}=1 024, 16 cabezas, máscara causal). Su secuencia de entrada concatena [32 latentes del *Perceiver Resampler*]+ [texto tokenizado]+[SOS]+[tokens de audio], y dos cabezas de salida predicen texto (*softmax* de 6 681) y audio (*softmax* de 1 026). **En entrenamiento** los tokens de audio de la entrada son los índices que produjo el DVAE sobre el audio real, de modo que basta un pase paralelo con *teacher forcing* y entropía cruzada contra esos mismos índices desplazados. **En inferencia** no existe audio real: el GPT muestrea los tokens uno a uno realimentando lo generado, hasta el token de fin o los 605 pasos (∼28 s). La salida para el sigiuente flujo del pipeline (lo que se entrega al vocóder) no son los índices discretos sino los *estados ocultos* de 1 024-d, que conservan la información continua que la cuantización descarta (una vez generado el token de fin se ejecuta todo el GPT y se toman los embeddings de la última capa).

5. **Decoder HiFi-GAN.** El vocóder (26 M parámetros) que devuelve la onda. A diferencia del HiFi-GAN clásico [4], que parte de un mel, aquí recibe directamente los estados ocultos (embedding de 1024 por token) de la secuencia de audio de GPT y sintetiza la onda completa a 24 kHz en una *sola* pasada no autorregresiva, mediante convoluciones transpuestas que acumulan un *upsampling* de 1 024 muestras por token. El *embedding* de locutor de 512-d se inyecta en cada etapa del generador de la GAN, siendo aquí el otro lugar donde se condiciona la generación con la identidad. La Hifi-GAN es preentrenada y solamente se usa en inferencia.

La Tabla II recoge los tamaños internos de todas estas representaciones y la Sección III-D contrasta qué módulos se activan en entrenamiento y en inferencia.

Tabla II
REPRESENTACIONES INTERNAS DE XTTS V2

Representación	Tamaño / #features	Longitud máx.
Tokens de texto (BPE)	vocab. 6 681	402 tokens
Latente de cond. (Perceiver)	32 × 1024	fijo, indep. de la ref.
Embedding de locutor	512 (H/ASP)	—
Mel interno	80 bandas	—
Token de audio (DVAE)	8 192→1 024 filtrado (+2)	605 tok. (∼28 s)
Tasa de tokens	21.53 Hz (1 024 muestras)	—
Estado oculto del GPT	1 024 (30 capas)	—
Onda de salida	1 canal	24 kHz

III-C. Fine-tuning liviano

Para el motor `xtts_ft` se afina *únicamente* el módulo GPT partiendo del *checkpoint* preentrenado —módulo que incluye el *Conditioning Encoder* y el *Perceiver Resampler*, que por tanto también reciben gradiente—; el DVAE, el codificador de locutor H/ASP y el HiFi-GAN quedan congelados. Se entrena 15 épocas sobre los 442 fragmentos de `train` (44.6 min) con tamaño de lote 16, optimizador AdamW y tasa de aprendizaje 5×10^{-6} con planificador `MultiStepLR` ($\times0.5$ en las épocas 7 y 12).

Afinar solamente GPT es el estándar oficial de fine-tuning del modelo y esta justificado al analizar la arquitectura: es el único bloque que modela la relación del texto con el audio (pronunciación y demás conceptos de vocalización).*prosodia y pronunciación*. Se supervisa el fine tuning con es entropía cruzada sobre los tokens de audio y de texto. La elección del resto de hiperparámetros (`max_conditioning_length`, longitudes de onda y texto, etc.) sigue el *recipe* oficial de Coqui [15] para no salir del régimen en el que el modelo fue preentrenado.

Como la pérdida optimizada es simbólica y no perceptual (Sección III-D), el *checkpoint* definitivo no se elige por mínima pérdida de validación: se guardan *checkpoints* cada dos épocas y se realiza un *barrido* sintetizando 30 frases de `val` con cada uno, eligiendo el mejor según las métricas objetivas de la Sección III-F.

III-D. Flujo de entrenamiento e inferencia

Los dos regímenes difieren no solo en si hay retropropagación, sino en *qué entra* al modelo y *qué módulos se activan*. La Tabla III lo resume, la Fig. 3 muestra cómo se organiza la secuencia dentro del GPT en cada caso, y a continuación se detalla.

Entrenamiento (fine-tuning). La entrada es un par (audio real, texto); no hay referencia externa: el condicionamiento (prefijo de GPT) se recorta (3–6 s) del *mismo* enunciado (voz). El DVAE congelado convierte el audio real en la secuencia objetivo y el GPT, con máscara causal y *teacher forcing*, predice todas las posiciones en *un único pase paralelo*. La pérdida actualiza *solo* el GPT. Durante el *fine-tuning* no se genera ni una muestra de onda ni existe pérdida alguna en el dominio del audio, todo se limita a la pérdida y aprendizaje de GPT para relacionar el texto con el audio tokenizado.

Inferencia. No existe audio real del enunciado objetivo. El papel de condicionante (condicionamiento en GPT y luego en

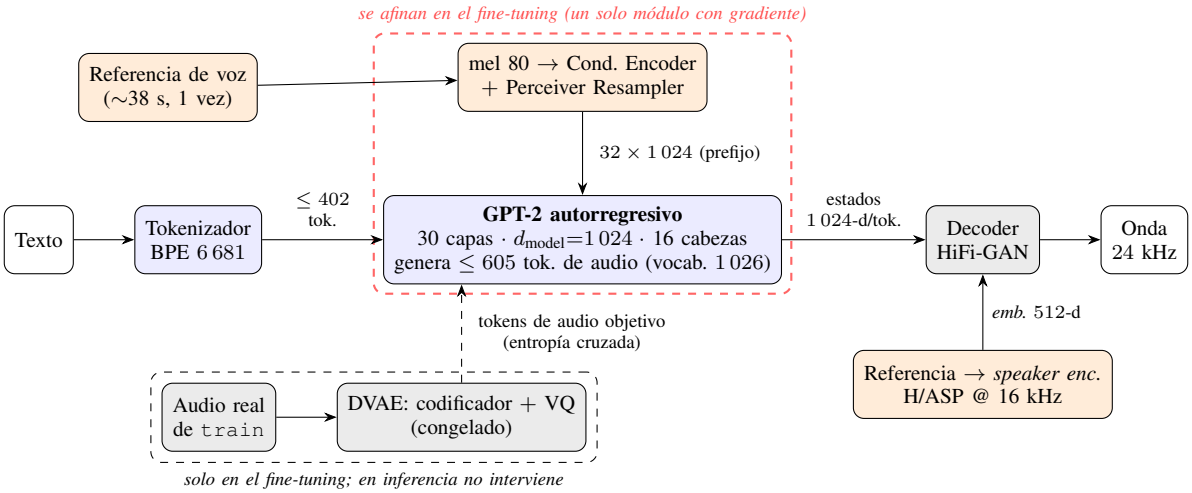


Figura 2. Arquitectura y flujo *texto* → *voz* de XTTS v2 (azul: módulos que procesan la entrada; naranja: condicionamiento de locutor; gris: congelados).

Tabla III
ENTRADAS Y MÓDULOS ACTIVOS

	Entrenamiento (*fine-tuning*)	**Inferencia**
Entradas	audio real + su texto	texto + audio de referencia
Condicionamiento	recorte de 3–6 s del *mismo* audio	referencia dedicada (∼38 s)
Tokens de audio	del DVAE (codificador+VQ) sobre el audio real	generados por el propio GPT
Paso por el GPT	1 pase paralelo, *teacher forcing*	N pasos secuenciales, autorregresivo
DVAE	codificador+cuantizador, congelado	no se usa
HiFi-GAN	no interviene	generador, congelado
Backward	CE audio (1.0) + texto (0.01) → solo GPT	no hay
Salida	un escalar de pérdida	onda a 24 kHz

GAN) lo asume la *grabación de referencia*, codificada una sola vez en los 32 latentes (GPT) y el *embedding* de 512-d (vocóder).

III-E. El marco Coqui TTS

Coqui TTS [15] es el *toolkit* de PyTorch del que procede XTTS: los modelos comparten interfaz y configuración declarativa (el `config.json` contra el que se verificaron los valores de la Tabla II) y un entrenador genérico. De ahí las dos ventajas prácticas de este trabajo: el envoltorio `GPTTrainer` reduce el *fine-tuning* a construir una configuración y llamar al entrenador, y la misma API de inferencia instancia el modelo *zero-shot* y el afinado con idéntica firma, lo que hace justa la comparación entre `xtts` y `xtts_ft`. Conviene no confundir XTTS con sus vecinos de librería VITS/YourTTS [16], [17]: aquellos son autocodificadores variacionales con predictor de duración explícito; en XTTS la duración es *implícita* (el GPT decide cuándo parar emitiendo el token de fin) y solo comparten el decodificador HiFi-GAN.

III-F. Métricas de evaluación

Cada texto de `test` se sintetiza con los tres motores y se compara con el audio original mediante:

- **Similitud de locutor** (↑): coseno entre *d-vectors* [13] del original y el reconstruido. ¿Suena a la misma persona? Se usa un modelo distinto (Resemblyzer) del ECAPA empleado en el filtrado, para no evaluar con el mismo criterio con el que se limpió el corpus.
- **WER / CER *round-trip*** (↓): el audio reconstruido se retranscribe con Whisper [11] y se compara con el texto de entrada. ¿Se entiende lo mismo?
- **MCD** (↓): distorsión mel-cepstral [14] con alineamiento DTW. ¿Se parece acústicamente?
- **UTMOS** (↑): naturalidad (MOS 1–5) *predicha* por un modelo sin referencia [19]. Capta artefactos de generación (cortes, prosodia plana) que las otras tres no ven; se calcula también sobre las grabaciones reales como nivel de referencia del canal.

La evaluación se restringe a `test` para que la comparación sea justa también con `xtts_ft`, cuyo `val` intervino en la selección del *checkpoint*.

IV. RESULTADOS Y ANÁLISIS

La Tabla IV presenta las métricas medias sobre 72 fragmentos de prueba por motor. La Fig. 4 muestra la evolución de la pérdida durante el *fine-tuning*.

- **La clonación es real.** Los motores que usan la referencia (XTTS y XTTS-FT) obtienen una similitud de locutor muy superior a Bark (0.669), que la ignora. Como Bark parte de un

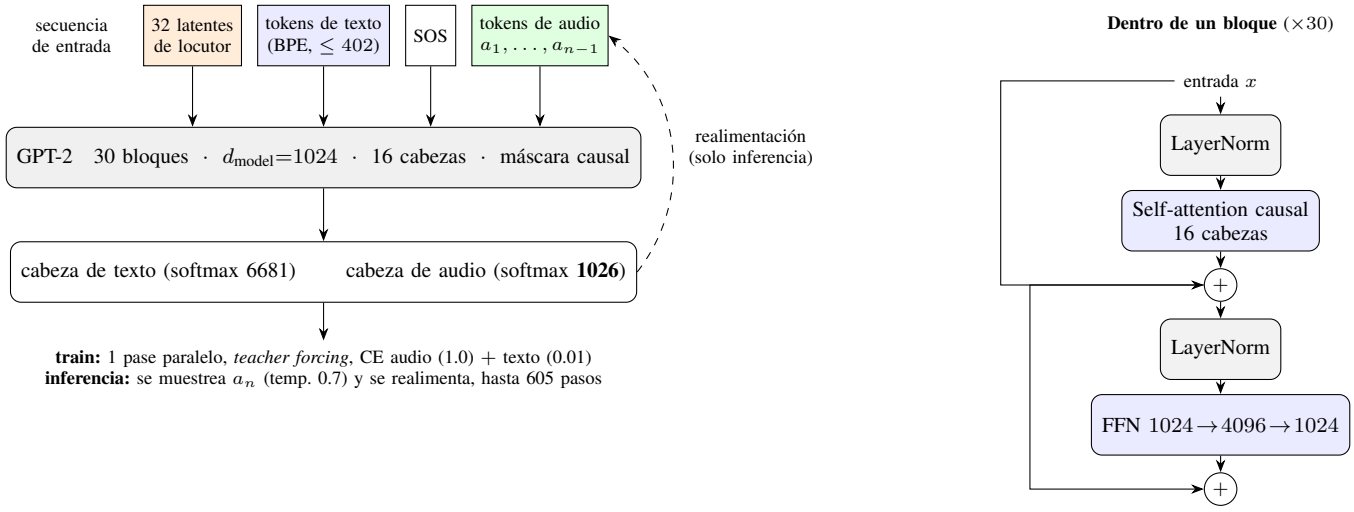


Figura 3. Interior del GPT de XTTS

Tabla IV
RESULTADOS SOBRE TEST. ↑/↓: MAYOR/MENOR ES MEJOR

Motor	spk_sim ↑	WER ↓	CER ↓	MCD ↓	UTMOS ↑
Bark (*baseline*)	0.669 ±0.04	0.056	0.040	10.07 ±1.59	2.52 ±0.22
XTTS (*zero-shot*)	0.804 ±0.04	**0.039**	**0.021**	10.40 ±1.06	**3.17** ±0.32
XTTS-FT (*fine-tuned*)	**0.859** ±0.03	0.076	0.047	**8.59** ±1.03	2.86 ±0.26



Figura 4. Pérdida de *train* y *val* durante el *fine-tuning* del GPT (15 épocas)

preset genérico, su baja similitud actúa de *control*: demuestra que el 0.80–0.86 de los motores XTTS proviene de imitar realmente la voz objetivo y no de un sesgo de la métrica del coseno.

- **El fine-tuning gana en identidad.** El *fine-tuning* eleva la similitud de locutor de 0.804 a 0.859 y reduce el MCD de 10.40 a 8.59. El análisis *emparejado* por fragmento (bootstrap con 10^4 remuestreos) es más contundente que las medias: Δspk_sim $= +0.055$, IC 95 % $[+0.049, +0.061]$, y ΔMCD $= -1.81$, $[-2.04, -1.58]$, ambos claramente significativos. Es un resultado notable considerando que el *fine-tuning* usó solo 44.6 min de audio ruidoso de Zoom, sin datos de estudio.
- **Un compromiso identidad–inteligibilidad.** El WER medio de XTTS-FT (0.076) es mayor que el de XTTS *zero-shot* (0.039; Δ pareado $+0.037$, $[+0.008, +0.070]$). Dos factores lo explican. Primero, las medias están infladas por la cola: el percentil 90 sube de 0.12 a 0.19 y un 1.4 % de síntesis son catastróficas (WER > 0.5), mientras que las *medianas* son casi perfectas (0.036 para XTTS-FT, 0.000 para XTTS). Segundo, al afinar sobre habla espontánea con muletillas, prosodia irregular y etiquetas ruidosas del STT, el modelo reproduce con más fidelidad el *estilo* del locutor —incluidas sus vacilaciones— a un pequeño coste de inteligibilidad literal. Para el objetivo del proyecto (recuperar la *identidad*) el intercambio es claramente favorable.
- **UTMOS delata el canal, no la identidad.** La naturalidad predicha ordena los motores al revés: XTTS *zero-shot* (3.17) supera a XTTS-FT (2.86; Δ pareado -0.31) y ambos superan a las *grabaciones reales* (UTMOS 2.47, por el ruido y la compresión de Zoom). Es la asimetría de la Sección III-D en acción: el *zero-shot* produce una señal "de estudio" más limpia que el propio original, y el *fine-tuning* arrastra la síntesis hacia el canal del corpus al adaptar identidad y canal a la vez.
- **El MCD por sí solo engaña.** El MCD *zero-shot* de XTTS (10.40) es peor que el de Bark (10.07) pese a que XTTS clona y Bark no: el MCD premia la envolvente espectral, que una voz genérica puede acertar por azar, y solo el *fine-tuning* (8.59) lo baja de forma consistente. De ahí la necesidad de *triangular* varias métricas.

V. CONCLUSIONES, LIMITACIONES Y MEJORAS

- **Hallazgos clave.** (i) Un *pipeline* totalmente automático convierte una grabación cruda de reunión en un corpus etiquetado apto para clonación. (ii) XTTS v2 *zero-shot* recupera una identidad de locutor sustancial (0.804) incluso con audio adverso. (iii) Un *fine-tuning* liviano del GPT (∼45 min de audio, *checkpoint* de la época 10 elegido por métricas) mejora la identidad de forma consistente y significativa (Δspk_sim $+0.055$, ΔMCD -1.81, pareados) a un coste moderado de

inteligibilidad y naturalidad, al adaptarse también al canal ruidoso. (iv) La comparación con Bark confirma que la ganancia es clonación real.

- **Limitaciones.** Las etiquetas provienen de un STT (*ruido de etiqueta*) y el WER *round-trip* usa el *mismo* Whisper que las generó: mide *consistencia* del ciclo, no exactitud absoluta. Los valores de MCD solo son comparables *dentro* de este estudio. El corpus es de un único locutor, un solo idioma y una única grabación ($\sim$58 min útiles con ruido de Zoom), por lo que la generalización no está garantizada. No se realizó evaluación subjetiva con oyentes: UTMOS es solo un sustituto aprendido del MOS. Bark, con *preset* genérico, no es un competidor justo de timbre.

- **Mejoras futuras.** Ampliar datos o generar variantes por *data augmentation* para reducir el sobreajuste; realizar una evaluación subjetiva (MOS) con oyentes; *des-sesgar* el WER re-transcribiendo con un STT distinto; buscar sistemáticamente los hiperparámetros de inferencia (temperatura, *top-k/p*, longitud de condicionamiento), que separan una buena clonación de una "voz robot"; estudiar la longitud y calidad óptimas de la referencia; añadir motores como Tortoise [9] u OpenVoice y evaluar síntesis multilingüe con la misma referencia; y aplicar realce/denoising al audio de Zoom antes de entrenar.

- **Consideración ética.** Que $\sim$40 s de referencia basten para clonar una identidad de forma convincente subraya el riesgo de suplantación. Los experimentos en cuestión se realizaron sobre la voz con consentimiento del locutor.

REFERENCIAS

[1] E. Casanova *et al.*, "XTTS: a Massively Multilingual Zero-Shot Text-to-Speech Model," in *Proc. Interspeech*, 2024, pp. 4978–4982. arXiv:2406.04904.

[2] C. Wang *et al.*, "Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers," arXiv:2301.02111, 2023.

[3] J. Shen *et al.*, "Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions," in *Proc. IEEE ICASSP*, 2018, pp. 4779–4783.

[4] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," in *Proc. NeurIPS*, 2020.

[5] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," in *Proc. NeurIPS*, 2017.

[6] A. Jaegle *et al.*, "Perceiver: General Perception with Iterative Attention," in *Proc. ICML*, 2021.

[7] A. Radford *et al.*, "Language Models are Unsupervised Multitask Learners," OpenAI Tech. Report, 2019.

[8] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High Fidelity Neural Audio Compression," arXiv:2210.13438, 2022.

[9] J. Betker, "Better Speech Synthesis through Scaling," arXiv:2305.07243, 2023.

[10] Suno AI, "Bark: Text-Prompted Generative Audio Model," https://github.com/suno-ai/bark, 2023.

[11] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proc. ICML*, 2023. arXiv:2212.04356.

[12] Silero Team, "Silero VAD: Pre-trained Enterprise-grade Voice Activity Detector," https://github.com/snakers4/silero-vad, 2021.

[13] L. Wan, Q. Wang, A. Papir, and I. L. Moreno, "Generalized End-to-End Loss for Speaker Verification," in *Proc. IEEE ICASSP*, 2018. arXiv:1710.10467.

[14] R. Kubichek, "Mel-cepstral Distance Measure for Objective Speech Quality Assessment," in *Proc. IEEE Pacific Rim Conf.*, 1993, pp. 125–128.

[15] E. Gölge and The Coqui TTS Team, "Coqui TTS," https://github.com/coqui-ai/TTS, 2021.

[16] J. Kim, J. Kong, and J. Son, "Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech," in *Proc. ICML*, 2021.

[17] E. Casanova, J. Weber, C. Shulby, A. C. Júnior, E. Gölge, and M. A. Ponti, "YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for Everyone," in *Proc. ICML*, 2022.

[18] B. Desplanques, J. Thienpondt, and K. Demuynck, "ECAPA-TDNN: Emphasized Channel Attention, Propagation and Aggregation in TDNN Based Speaker Verification," in *Proc. Interspeech*, 2020, pp. 3830–3834.

[19] T. Saeki, D. Xin, W. Nakata, T. Koriyama, S. Takamichi, and H. Saruwatari, "UTMOS: UTokyo-SaruLab System for VoiceMOS Challenge 2022," in *Proc. Interspeech*, 2022, pp. 4521–4525.